



KTH Technology and Health

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	Onsdag 2021-10-27 08:00-13:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt minst hälften av poängen på B-delen och att minst en av de två första programmeringsuppgifterna är helt korrekt lösta. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>När du skapar projektet i IntelliJ eller NetBeans måste du kontrollera att projektet hamnar under VPROVXXXX/H, ditt tentamenskonto. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg1, uppg2, ...</i> Innan du avslutar tentamen: Kontrollera att du verkligen sparar dina lösningar under VPROVXXXX/H

För att komma åt måsvingar, hakparenteser och "|", använd "AltGr" + lämplig tangent,
t.ex. "AltGr" + "{" (7:ans tangent).

Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. Motivera alltid dina svar.

Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

1. Vilka synlighetsnivåer, "visibility", finns i Java och vad innebär var och en av dessa?
(2 p)
2. Förklara, i kontexten objektorienterad programmering, vad som menas med begreppet "låg koppling" samt vad poängen med detta är. Notera att frågan inte gäller "hög kohesion".
(2 p)
3. Vad är en enum i Java? Vad är poängen med att använda enums istället för konstanter (t.ex. deklarerade som `public static final int FOO = 42`)?
(2 p)
4. Antag att vi i en klass har en privat datamedlem, `x`, och en access-metod, `getX`. I vissa fall bör man returnera en kopia. Varför? I vilka fall behöver man aldrig skapa en kopia?
(2 p)
5. Förklara vad som, i Java, skiljer en abstrakt klass från ett interface vad gäller
 - datamedlemmar,
 - metoder,
 - arv.
(2 p)
6. Skriv kod för en metod som öppnar en fil, läser från denna och sedan garanterat stänger filen innan metoden avslutas. Metoden ska *inte* hantera eventuella fel som uppstår, det överlämnas till anropande metod. Ditt exempel behöver inte vara helt syntaktiskt korrekt när det gäller detaljer i att läsa från fil, men koden som relaterar till felhantering och att stänga resursen (här filen) ska vara korrekt.
(2 p)
7. Vad är en "event handler" i JavaFX och i vilken situation används en sådan? Vad behöver du som programmerare göra för att händelsehanteringen ska fungera?
Du behöver inte skriva kod utan endast ange de steg programmeraren behöver implementera.
(2 p)
8. Beskriv designmönstret Proxy med hjälp av ett klassdiagram (ev. kan även interface ingå).
Ange en situation där det är meningsfullt att använda designmönstret.
(2 p)

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot. För varje lösning ska det finnas ett paket med namnet "uppg1", "uppg2" etc. Kontrollera att du verkligen sparar dina lösningar på H.

Till dessa uppgifter kan du i vissa fall ha hjälp av Javas API-dokumentation, som finns på P:/Java/api. *Alla klasser ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha en konstruktör som initierar alla datamedlemmar, access-metoder samt en omdefinierad version av toString.*

1. Triangelkontrakt

Skriv en klass som representerar en triangel med sidorna (a, b och c) representerade som heltal. Klassen ska implementera interfacet ITriangle nedan.

```
public interface ITriangle {  
    void setA(int a) throws IllegalArgumentException;  
    void setB(int b) throws IllegalArgumentException;  
    void setC(int c) throws IllegalArgumentException;  
    int getA();  
    int getB();  
    int getC();  
    double getArea();  
}
```

I en giltig triangel är varje sida större än noll samt mindre än de två övriga sidorna tillsammans (triangelolikheten). Skriv en privat hjälpmetod för detta test. Konstruktorn och de tre set-metoderna ska kontrollera att värdena på a, b och c tillsammans representerar en giltig triangel, i annat fall ska ett exception, IllegalArgumentException kastas (du skriver klassen själv).

Metoden getArea ska returnera ett flyttal som beräknas som kvadratroten (sqrt) ur $s(s-a)(s-b)(s-c)$, där $s = (a+b+c)/2.0$. Notera att s måste beräknas till ett flyttal.

Förutom det ovan angivna ska klassen implementera den *generiska* versionen av interfacet java.lang.Comparable, där rangordningen bestäms av triangelns area, samt omdefiniera den equals-metod som ärvs från klassen Object. Notera att klassen alltså kommer att implementera två interface: ITriangle och Comparable.

För godkänt på denna uppgift måste du visa att du kan

- implementera inkapsling korrekt, så att triangelobjekt alltid har giltiga värden på sidorna,
- definiera en egen exception-klass och kasta exception då argument till metoder/konstruktör är felaktigt,
- implementera java.lang.Comparable och omdefiniera Object.equals korrekt.

(4 p)

2. Varor

I denna uppgift är det tillåtet att använda Javas implementationer av datastrukturer, t.ex. ArrayList.

En matvaruaffär säljer många olika typer av produkter och det är intressant att registrera (bland annat) namn, vikt och pris för dessa. En del produkter är sammansatta av andra produkter, t.ex. kan en lunchsallad vara sammansatt av pasta, grönsaker m.m. Pris och vikt är då summan av delprodukternas respektive priser och vikter.

Du ska skriva klasser som kan representera olika typer av produkter, inklusive en klass för den sammansatta produkten. Alla dessa klasser ska implementera interfacet IProduct, se nedan.

```
public interface IProduct {  
    String getName();  
    double getPrice();  
    double getWeight();  
}
```

Skriv en abstrakt klass, AProduct, som implementerar IProduct samt dessutom definierar konstruktör och lämpliga datamedlemmar (inga set-metoder). Skriv sedan subklasserna DairyProduct (mejerivara) och PastaProduct. Förutom namn, pris och vikt har en mejerivara ett bäst-före-datum (använd LocalDate). PastaProduct har en typ, spaghetti, lasagne etc, som representeras via en enum. Det finns mycket hjälp i IntelliJ för att implementera dessa klasser (högerklicka + Generate), men kontrollera att den genererade koden uppfyller villkoren ovan.

Sammansatta produkter representeras av klassen CompoundProduct. Denna klass implementerar IProduct direkt (utan att gå via AProduct). Det ska gå att lägga till och ta bort vilka produkter som helst som implementerar interfacet IProduct i en CompoundProduct, inklusive andra CompoundProducts. För detta ändamål skriver du metoderna add(...) och remove(...) med lämpliga parametrar. Metoderna getPrice och getWeight ska, i CompoundProduct, returnera totala priset respektive vikten. Det ska även finnas en metod som returnerar en kopia av den interna listan med produkter.

(4 p)

3. Lista

För poäng på denna uppgift måste lösningen vara korrekt implementerad med generics (typparameter). Du får *inte* använda existerande list-klasser från Javas API i din lösning.

Skriv en generisk listklass, TList, som implementerar nedanstående interface.

```
public interface IList<T> {  
    int size(); // number of elements in list  
    void add(T obj); // add at end of list  
    void addAll(IList<T> objects); // add at end of list  
    boolean contains(T obj);  
    T get(int index) throws java.lang.IndexOutOfBoundsException;  
    T remove(int index) throws java.lang.IndexOutOfBoundsException;  
}
```

Internt ska det finnas en privat array som håller objekten. Arrayen ska utökas när så behövs och packas när objekt tas bort, detta sköts via privata hjälpmetoder.

Förutom konstruktör är de enda publika metoder som ska finnas i TList de som anges i interfacet samt en omdefiniering av toString.

(4 p)

4. Funktionellt

Du ska skriva funktionalitet för att utifrån en lista med objekt skapa en ny lista som innehåller nya objekt som representerar någon egenskap hos de ursprungliga objekten. Exempelvis ska vi kunna anropa metoden med en lista av personer och få en ny lista med respektive persons födelsedatum.

För detta behöver du skriva ett interface, Mapper, med en metod, apply, som tar ett enda objekt av som argument och returnerar ett nytt objekt, eventuellt av en annan typ. Interfacet måste alltså vara generiskt, med två typ-paramterar, en för argumentets typ och en för returtypen.

Metoden för att konvertera en hel lista till en ny lista med de nya objekten, mapToList, skrivs som en klassmetod, i en separat klass¹. Även i detta fall ska koden vara generisk.

Skapa sedan de två implementationerna av interfacet som beskrivs nedan.

- En implementation som tar en textsträng och returnerar en Boolean som indikerar om strängen är kort, t.ex. färre än två tecken.
- En implementation som tar ett DairyProduct-objekt, från uppgift 2, och returnerar bäst-före-datum.

Skriv sedan en main-metod där du skapar en lista av texter respektive en lista av DairyProduct-objekt och konverterar med mapToList och passande interface-implementation. I main-metoden kan det se ut som nedan².

```
List<String> words =  
    Arrays.asList(new String[]{"to", "code", "or", "not", "to", ...});  
  
List<Boolean> isShortStrings =  
    mapToList(words, new IsShortStringMapper(2));
```

(4 p)

¹ Metoden bör alltså vara en klassmetod, static, men om du inte får till detta kan du skriva en vanlig metod, dock fortfarande i en separat klass.

² Om du löser uppgiften korrekt kan du även använda lambda-uttryck, istället för uttryckliga interface-implementationer, som argument till mapToList (du behöver dock inte demonstrera detta).